

Neural Networks

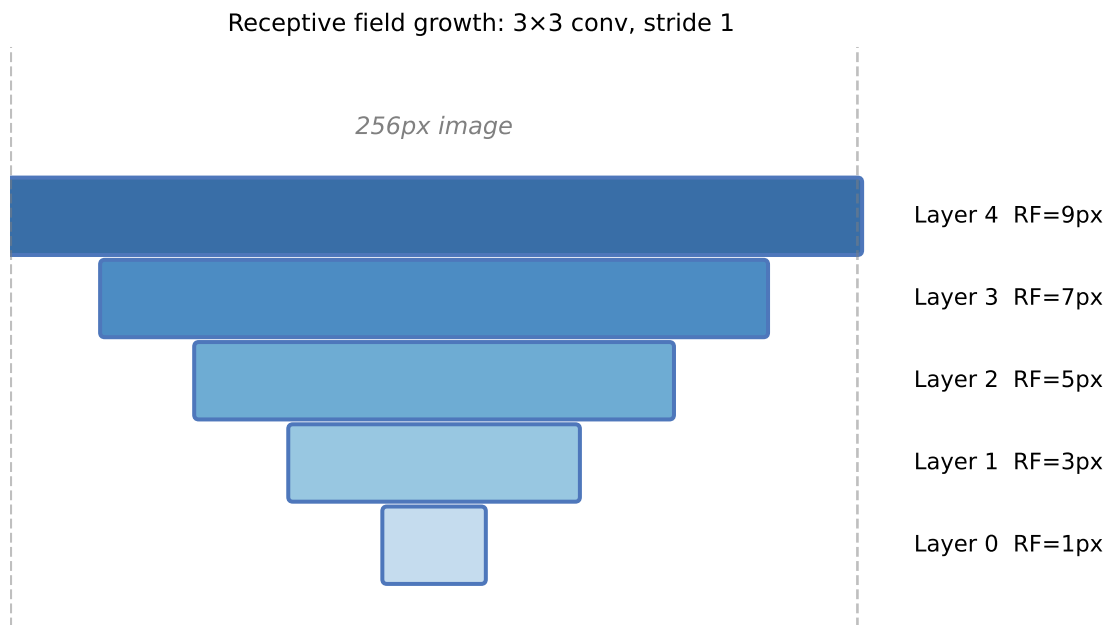
05 — Modern Architectures & Beyond CosmicAI Summer School 2026

The CNN limit

A 3×3 filter sees only its immediate neighbourhood. After N layers, the receptive field grows by 2 pixels per layer.

For a 256×256 image: **127 layers** before opposite sides can interact.

- Spiral arms on opposite sides of a galaxy can't be compared until very deep
- Global shape (ellipticity, PA) is hard to capture
- Long-range dependencies need a different mechanism



Self-attention: the key idea

Instead of a fixed local filter, let each position query every other position.

Queries, Keys, Values:

- **Query** Q : “what am I looking for?”
- **Key** K : “what do I contain?”
- **Value** V : “what do I output if matched?”

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Every output position attends to every input position — in **one operation**.

```
class SelfAttention(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.scale = d_model ** 0.5

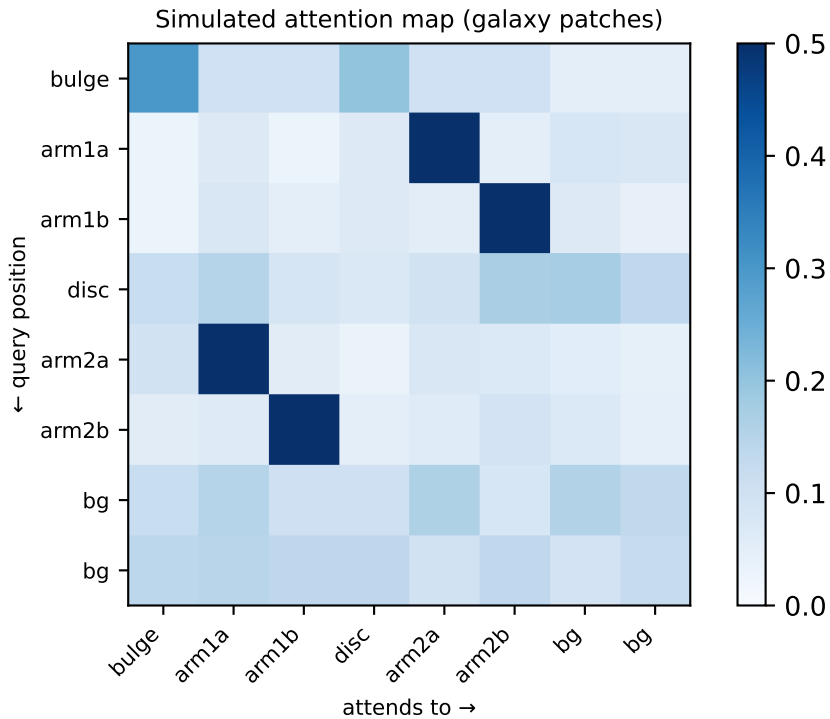
    def forward(self, x):
        Q = self.W_q(x)
        K = self.W_k(x)
        V = self.W_v(x)
        scores = (Q @ K.transpose(-2, -1)) / self.scale
        weights = scores.softmax(dim=-1) # attention map
        return weights @ V
```

What attention computes

The attention weight a_{ij} tells you: **how much does position i attend to position j ?**

For galaxy morphology: - A spiral arm on one side can directly attend to the arm on the other
- The bulge can attend to the full disc - No stacking of 127 layers required

The $\sqrt{d_k}$ scaling prevents softmax saturation (vanishing gradients).



Vision Transformer (ViT): CNNs meet attention

What if we treat an image as a sequence of patches, and apply attention to patches?

ViT procedure:

1. Split image into 16×16 pixel patches
2. Flatten + project each patch to a vector (the “token”)
3. Add a **[CLS] token** (learnable; represents the whole image)
4. Add **positional embeddings** (the model has no other notion of position)
5. Pass token sequence through transformer encoder blocks
6. Use [CLS] token output to classify

```
class PatchEmbedding(nn.Module):
    def __init__(self, img_size=256,
                  patch_size=16, d_model=768):
        super().__init__()
        n_patches = (img_size // patch_size) ** 2
        self.proj = nn.Linear(
            3 * patch_size * patch_size,
            d_model
```

```

)
self.cls = nn.Parameter(
    torch.randn(1, 1, d_model))
self.pos = nn.Parameter(
    torch.randn(1, n_patches + 1, d_model))

```

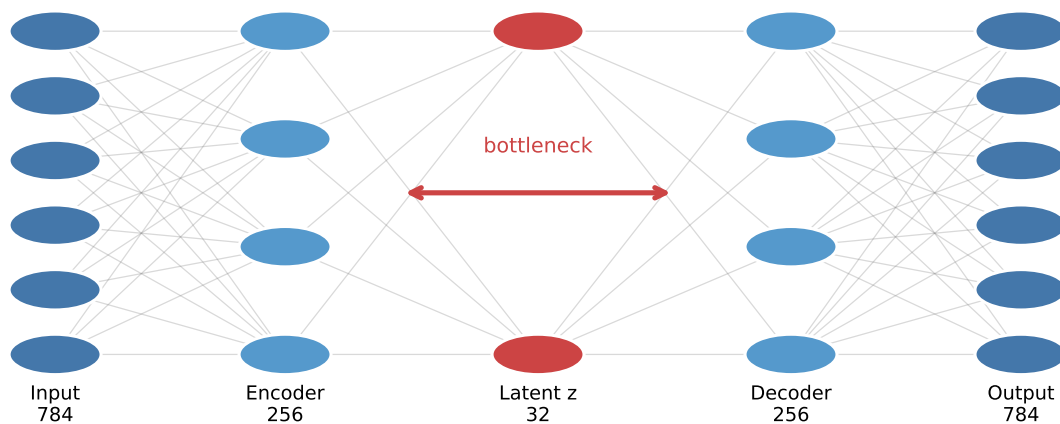
Autoencoders: learning compressed representations

An **autoencoder** forces the network to compress data into a low-dimensional latent space, then reconstruct it.

The latent code z is the useful part — used for clustering, anomaly detection, or generation.

Workshop 3 uses a pre-trained encoder as a feature extractor for ResNet fine-tuning.

Autoencoder: compress → reconstruct



Practical guidance: when to use which architecture

i Note

Start simple. Add complexity only when you have evidence you need it.

Problem	First choice	When to go further
Tabular / catalogue data	MLP	Rarely needed

Problem	First choice	When to go further
Fixed-size images, position-invariant	CNN	GPU available, dataset $> \sim 5k$
Long-range dependencies in images	CNN + attention	Large dataset, competitive task
Sequence (spectra, time series)	1D CNN or Transformer	$> \sim 10k$ labelled examples
Small labelled dataset	Pre-trained CNN (fine-tune)	Always — see Workshop 3

Warning

A ViT trained from scratch needs $\sim 1M+$ images to outperform a CNN. With 5,000 labelled galaxies, a well-tuned CNN is almost always better.

Astronomy-specific considerations

Rotational invariance

Galaxies can appear at any position angle. CNNs are translationally invariant but *not* rotationally invariant.

Options: - Data augmentation: random rotation during training - Equivariant architectures (e.g. E(2)-CNNs) - Accept it and train on enough orientations

Class imbalance

Spiral galaxies outnumber ring galaxies by 20:1 in most surveys.

Options: - Weighted cross-entropy loss - Oversampling rare classes - Report per-class F1, not just accuracy

Warning

A classifier that always predicts “spiral” achieves $\sim 70\%$ accuracy on GalaxyZoo. Accuracy alone is a misleading metric here.

Astronomy applications

Task	Architecture	Reference
Galaxy morphology (GalaxyZoo 2)	CNN + transfer learning	Dieleman et al. (2015)
Pulsar candidate classification	CNN on fold profiles	Luna et al. (2021)
Strong lens detection	CNN, ResNet	Jacobs et al. (2019)
Photometric redshift	MLP / CNN on multi-band cutouts	D’Isanto & Polsterer (2018)
Radio transient classification	CNN + attention	Farah et al. (2019)
Spectral line detection	1D CNN	Hattori et al. (2023)

Note

In all these cases: start with a published CNN baseline before reaching for transformers. The architecture is rarely the bottleneck — data quality and quantity usually are.

Case study: pulsar candidate classification

Luna et al. (2021) — classifying pulsar candidates from the HTRU survey

Input: 4-panel diagnostic plot per candidate - Profile (folded pulse shape) - Sub-band profile - Sub-integration profile - DM–time plot

Model: CNN classifying each panel separately, outputs merged

Result: 99.2% recall on confirmed pulsars, 95% precision — compared to 87% recall for manual inspection

Key lessons from this paper: - Human-interpretable features (pulse width, DM) helped design the CNN input - Class imbalance was severe: ~1 pulsar per 1000 candidates - Weighted loss + oversampling both needed - The model generalises to new observations **only** if trained on similar telescope systems

Want to go further?

The bonus notebook covers the topics dropped from this deck in a hands-on, self-paced format.

Note

bonus_generative_models.ipynb — open any time after the school:

- **Part 1:** Variational Autoencoders — explore the latent space interactively

- **Part 2:** Generative Adversarial Networks — watch the generator/discriminator dynamics
- **Part 3:** Diffusion models — step through the forward noising process
- **Part 4:** Contrastive / self-supervised learning — see how representations cluster

No GPU required. No exercises — purely exploratory.

These architectures are increasingly used in astronomy: VAEs for anomaly detection, GANs for data augmentation, diffusion for simulation-based inference.

References

Papers

- Vaswani et al. (2017) — *Attention Is All You Need*. NeurIPS. arxiv.org/abs/1706.03762
- Dosovitskiy et al. (2020) — *An Image is Worth 16×16 Words*. ICLR. arxiv.org/abs/2010.11929
- Luna et al. (2021) — *Pulsar candidate classification using CNNs*. arxiv.org/abs/2102.10326

Blogs

- Colah — *Attention and Augmented RNNs* distill.pub/2016/augmented-rnns/
- Colah — *Understanding LSTM Networks* colah.github.io/posts/2015-08-Understanding-LSTMs/